

KSP - CIAP

Crime Intelligence & Analytical Platform

Technical Architecture & Methodology

Geospatial hotspot detection · graph-based link analysis · predictive risk scoring

Prepared as a full-stack prototype for Karnataka State Police

1. Executive Summary

Most policing dashboards in use today are static reporting tools — they summarize what already happened, days or weeks after the fact, from siloed district spreadsheets. CIAP inverts that model. It is a dynamic intelligence engine that ingests district-level incident data continuously, fuses it with a criminal-network graph, and produces a live, explorable picture of where and when crime is concentrated, who is connected to whom, and which districts are trending into anomalous territory before the trend becomes a pattern anyone had to notice manually.

The prototype is organized around four capabilities that map directly to how an analyst actually works a case: **see where** (geospatial hotspots), **see who** (link analysis), **see what's coming** (predictive risk), and **act on it** (role-gated alerting).

2. System Architecture

The system is split into four layers, each independently deployable and independently scalable.

Layer	Technology	Responsibility
Presentation	React 18 + Recharts + custom SVG force-graph	Interactive dashboard: hotspot map, link-analysis graph, predictive panel, alert feed
API	FastAPI (Python), JWT auth, Pydantic v2 schemas	Authentication, permission checks, request validation, OpenAPI-documented REST endpoints
Relational + spatial store	PostgreSQL + PostGIS	Incidents, districts, stations, users — indexed with GIST for fast radius/bounding box queries
Graph store	Neo4j	Suspect-location-incident graph; shared-location and MO-cluster traversal queries
Predictive layer	Scikit-learn (IsolationForest) scheduled job	Computes a 0–100 risk score per district per window, flags statistical anomalies

Data flow

- 1 District stations drop raw incident data as Excel/CSV files (existing workflow — no change required in the field).
- 2 A Pandas ingestion job cleans and normalizes the data: standardizes crime-type taxonomy, parses FIR numbers, geocodes missing coordinates, deduplicates.
- 3 Cleaned records are upserted into PostgreSQL/PostGIS; the suspect/location/incident subset is mirrored into Neo4j as graph nodes and relationships.
- 4 A scheduled job recomputes a risk score per district from recent incident volume vs. rolling baseline, using IsolationForest for anomaly flagging.
- 5 The FastAPI layer exposes all of the above as versioned REST endpoints, gated by role-based permissions.
- 6 The React dashboard polls/queries these endpoints and renders the hotspot map, link graph, risk gauges, and alert feed.

3. Methodology by Capability

3.1 Geospatial Hotspot Engine

Incidents are stored as PostGIS **POINT(4326)** geometries with a GIST index, so density queries for a bounding box or radius run in milliseconds even at millions of rows. The dashboard's 24-hour time-slider buckets incidents by hour-of-day, letting an analyst scrub through a full day and watch density shift — e.g. property crime clustering around transit hubs in the evening, assault clustering near nightlife districts after 11pm. This is the difference between a static crime map and an actual patrol-planning tool.

3.2 Graph-Based Link Analysis

Suspects, locations, and incidents are modeled as a property graph in Neo4j rather than SQL join tables, because the actual analyst question — *"find everyone who has been at the same location as Suspect X in the last 30 days"* — is a graph traversal, not a join. In SQL this query gets expensive fast as the relationship depth grows; in Cypher it stays a single readable pattern match. A nightly job also derives weighted **ASSOCIATE_OF** edges between suspects who share two or more locations, surfacing informal networks that were never explicitly recorded anywhere.

3.3 Predictive Risk Scoring

Each district's incident volume is compared against its own rolling baseline using an IsolationForest model — chosen because it doesn't assume a normal distribution, handles the seasonal and day-of-week swings typical of crime data, and flags anomalies without needing hand-tuned thresholds per district. The output is a 0–100 risk score plus a boolean anomaly flag, recomputed on a schedule so the predictive panel reflects the current window, not a stale snapshot.

3.4 Role-Based Alerting

Three roles — Field Officer, Intelligence Analyst, Administrator — map to escalating permission sets (view → view + annotate + export → view + annotate + export + assign + dismiss). Every alert and graph mutation endpoint checks permissions server-side via a FastAPI dependency, not just in the UI, so the RBAC boundary can't be bypassed by calling the API directly.

4. API Summary

Endpoint	Method	Purpose	Min. role
/api/v1/auth/login	POST	Badge number + password -> JWT access token	-
/api/v1/auth/me	GET	Current user identity + effective permissions	officer
/api/v1/incidents	GET	Paginated incidents, filter by district / time window	officer
/api/v1/incidents/hotspots	GET	Per-district incident counts, optional hour-of-day bucket	officer
/api/v1/risk/districts	GET	Latest computed risk score per district	officer
/api/v1/risk/recompute	POST	Trigger the anomaly-detection job on demand	analyst
/api/v1/alerts	GET	Active + historical alerts	officer
/api/v1/alerts/{id}/ack	POST	Acknowledge an alert	admin
/api/v1/districts	GET	Reference list of districts	officer
/api/v1/suspects	GET	Reference list of seeded suspects	officer
/api/v1/graph/neighbors/{name}	GET	Shared-location suspect neighbors (SQL now, Neo4j at scale)	officer
/api/v1/ingest/csv	POST	Clean + upsert pre-parsed CSV rows, dedupes on FIR number	admin
/health	GET	Liveness/readiness probe for load balancers	-

Full interactive documentation is auto-generated by FastAPI at **/docs** (Swagger UI) and **/redoc** from the same Pydantic schemas — no hand-maintained API reference to fall out of sync.

5. Security & Access Control

- Passwords hashed with PBKDF2-HMAC-SHA256, 100,000 iterations, unique salt per user — no plaintext or reversible storage.
- Stateless JWT bearer tokens (2-hour expiry by default) — no server-side session store to manage or leak.
- Every mutating and role-sensitive endpoint enforces permissions server-side via a FastAPI dependency, independent of what the UI shows.
- CORS origins are explicitly allow-listed per environment rather than left open in production.
- Rate limiting on the login endpoint mitigates credential-stuffing attempts.
- Secrets (JWT signing key, DB credentials) are read from environment variables, never hardcoded — ready for a secrets manager in production.

6. Deployment Overview

The backend runs unmodified on SQLite for local development and demos, and on PostgreSQL/PostGIS in production via a single environment variable (`DATABASE_URL`). It ships with a Dockerfile and a docker-compose stack (API + Postgres/PostGIS + Neo4j) for a one-command local environment that mirrors production topology. See **DEPLOYMENT.md** for step-by-step instructions covering Railway/Render, a Docker VPS, AWS ECS Fargate, and Google Cloud Run.

Target	Effort	Best for
Railway / Render	Lowest	Fastest path to a live demo URL for a portfolio or pitch
Docker + VPS	Low-medium	Full control on a budget, e.g. DigitalOcean droplet
AWS ECS Fargate + RDS	Medium	Production-grade, auto-scaling, matches enterprise procurement expectations
Google Cloud Run	Low	Serverless, scales to zero, pay-per-request

7. Roadmap

Phase	Scope
Now (prototype)	Interactive dashboard, RESTful API on SQLite/Postgres, mock predictive scoring, RBAC
Next	Wire live Neo4j link-analysis queries, real IsolationForest training on historical FIR data, Mapbox GL for true GIS
Later	WebSocket push for real-time alerts, mobile app for field officers, integration with existing state FIR systems, au

Files referenced in this document — CIAP_Dashboard.jsx, main.py, tests/test_api.py, schema_postgis.sql, neo4j_graph.cypher, Dockerfile, docker-compose.yml, requirements.txt, DEPLOYMENT.md — are included alongside this PDF.